🚀 Lecture 04.01: Strings in Python & Memory Immutability

📌 Executive Architectural Summary

A string in Python is an **immutable sequence of Unicode characters**. Under CPython, strings are represented by the `PyUnicodeObject` structure, which dynamically selects compact encoding representations (ASCII, Latin-1, UCS-2, or UCS-4) based on character ordinal values (PEP 393).

🔬 Core Engineering Mechanics

1. Immutability Invariant

Strings in Python are strictly immutable. Once a `str` object is allocated on the heap, its character payload cannot be modified in-place:

```
s = "Python"
s[0] = "J" # Raises TypeError: 'str' object does not support item assignment
```

Any modification operation (such as `s.replace()` or `s.upper()`) allocates a **completely new `str` object** in heap memory.

2. Zero-Based Indexing & Character Access

For a string S of length $N = |S|$:

- Forward indices span: $0, 1, 2, \ldots, N-1$
- Negative indices span: $-N, \ldots, -2, -1$ where index -1 maps to the last character $S[N-1]$.

💻 Production Python Implementation

```
# String Instantiation & Indexing
course = "Apna College AI/ML Prime"

print(f"String: {course}")
```

```
print(f"Length: {len(course)} characters")
print(f"First character (index 0): {course[0]}")
print(f"Last character (index -1): {course[-1]}")

# Demonstration of Immutability
original_id = id(course)
course_upper = course.upper()
print(f"Original ID: {original_id} ('{course}')")
print(f"New Object ID: {id(course_upper)} ('{course_upper}')")
```